

UNIT - 10 MOST IMP QUESTION (OFFLINE 5 MARKS)

WRITE A PROGRAM TO CREATE MODELS,SERIALIZERS,PERMISSIONS AND ROUTER FILE FOR A STUDENT MANAGEMENT APP

- Also write the steps to create the project `studentmanagement` and the app `students`
- Create a Student model with name, roll and city
- Perform migrations
- Create ModelForm for student entry
- Create StudentSerializer class to fetch all fields
- Add permission for the user to restrict the access
 - Student can view the records
 - Admin can perform CRUD operations
- Create `urls.py` file to configure routing of the app

Create Project and app

```
python -m django startproject studentmanagement
cd studentmanagement
python manage.py runserver (## This will create sqlite database file)
python manage.py startapp students (## Register the app in project settings)
```

Create Student model with fields `name, roll and city`

- `models` is used to create queryset for database tables

```
from django.db import models

class Student(models.Model):
    name = models.CharField(max_length=200)
    roll = models.IntegerField()
    city = models.CharField(max_length=100)
    def __str__(self):
        return self.name
```

Register the model in `admin.py`

```
from .models import Student
admin.site.register(Student)
```

Perform migrations (to create auth,session,user,app tables)

```
python manage.py makemigrations
python manage.py migrate
```

CREATE MODELFORM

- UserCreationForm is used to provide form framework to create a user
- forms.ModelForm is used to update the present user (i.e. name,roll or city)

```
from django import forms
from django.contrib.auth.forms import UserCreationForm
from django.contrib.auth.models import User
from .models import Student

class SignupForm(UserCreationForm):
    email = forms.EmailField(required=True)
    class Meta:
        model = Student
        fields = ['username', 'email', 'password1', 'password2']
class StudentForm(forms.ModelForm):
    class Meta:
        model = Student
        fields = ['name', 'roll', 'city']
```

CREATE STUDENTSERIALIZER TO FETCH ALL FIELDS

- serializers.py file is used to convert complex python datatypes to json and from json to complex python datatypes
- It is also used for validation purpose

students/serializers.py

```
from rest_framework import serializers
from .models import Student

class StudentSerializer(serializers.ModelSerializer):
    class Meta:
        model = Student
        fields = "__all__"
```

CREATE MODEL VIEWSET IN students/views.py

- It is used to create api logic for django rest framework

```
from rest_framework import viewsets
from rest_framework.permissions import IsAuthenticated
from .models import Student
from .permissions import IsAdminOrReadOnly
from .serializers import StudentSerializer
class StudentViewSet(viewsets.ModelViewSet):
    queryset = Student.objects.all()
    serializerclass = StudentSerializer
    permission_classes = [IsAuthenticated,IsAdminOrReadOnly]
```

students/permissions.py

- It is used to control and restrict the api access to authorized users

```
from rest_framework.permissions import BasePermission,SAFE_METHODS
class IsAdminOrReadOnly(BasePermission):
    def has_permission(self,request,view):
        if request.method in SAFE_METHODS:
            return True
        return request.user.is_staff
```

students/urls.py to configure routing

- It is used to configure routing to automatically create api paths

```
from django.urls import path,include
from .views import StudentViewSet
from rest_framework.routers import DefaultRouter

## Create router
router = DefaultRouter()
router.register(r'students',StudentViewSet,basename = 'students')
urlpatterns = [
    path('api/',include("router.urls"))
]
```

- router urls (GET,PUT,POST,PATCH,DELETE) are automatically created

CREATE views.py for the signup,login,logout,home (django auth) (5 marks)

```
from django.contrib.auth.decorators import login_required
from django.contrib.auth.forms import UserCreationForm
from django.contrib.auth import login,logout,authenticate
from django.shortcuts import render,redirect
```

```

from django.contrib import messages

### DASHBOARD #####
def dashboard(request):
    return render(request, 'students/dashboard.html')

#### SIGNUP #####
def signup(request):
    if request.method == "POST":
        form = UserCreationForm(request.POST)
        if form.is_valid():
            user = form.save()
            login(request, user)
            return redirect("dashboard")
    else:
        form = UserCreationForm()
    return render(request, 'students/signup.html', {'form': form})

##### LOGIN #####
def user_login(request):
    if request.user.is_authenticated:
        return redirect('dashboard')
    if request.method == "POST":
        username = request.POST.get('username')
        password = request.POST.get('password')
        user = authenticate(request, username=username, password=password)
        if user is not None:
            login(request, user)
            messages.success(request, "Logged in successfully")
            return redirect("dashboard")
        else:
            messages.error(request, "Invalid Credentials")
    return render(request, 'students/login.html')

##### LOGOUT #####
def custom_logout(request):
    logout(request)
    return render(request, 'students/logout.html')
#####

urls.py

from django.urls import path
from . import views
urlpatterns = [
    path("", views.dashboard, name='dashboard'),
    path("login/", views.user_login, name='login'),
    path('signup/', views.signup, name='signup'),
    path("logout/", views.custom_logout, name='logout')
]

```